

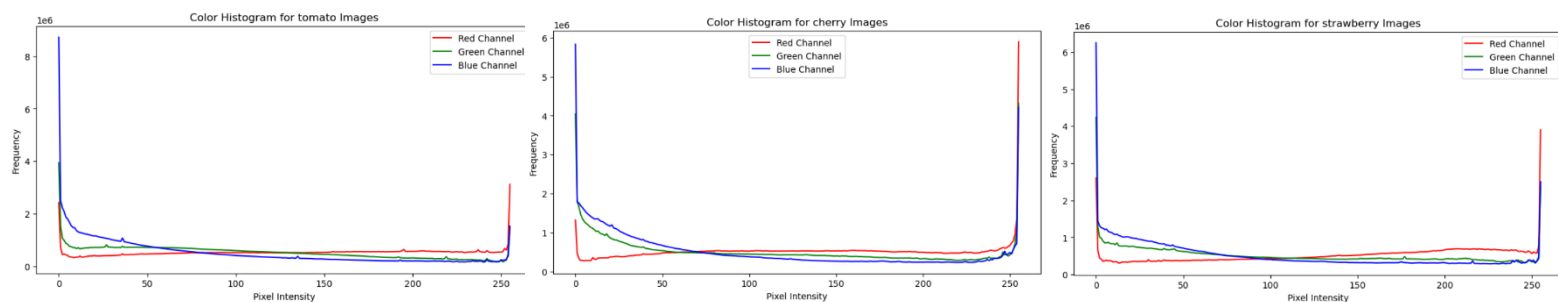
Introduction

The objective of this project is to classify images of strawberries, tomatoes, and cherries. These fruits may look visually distinct to the human eye but due to the subtle colour and shape variations it makes it challenging to differentiate programmatically. Convolutional Neural Networks (CNNs) emerged as a powerful tool for image classification due to their ability to learn spatial hierarchies in data. This report outlines the steps that have been taken to build, train, and evaluate a CNN model that classifies these images, as well as comparing it to a simpler Multilayer Perceptron (MLP) as a baseline model. Various pre-processing and feature extraction techniques were used to improve the classification performance as well as the exploration of the effects of data augmentation and transfer learning.

Problem Investigation

Exploratory Data Analysis (EDA)

The dataset contains 4500 images total with 1500 images for each tomato, cherry, and strawberry class. Through EDA, the following observations were made:



- **Colour Analysis:** The colour histograms above showed that all three classes show a high presence of red, which makes it challenging for a model to differentiate based mainly on colour. The low-intensity peaks in the blue channel across all three classes may indicate the background content or shadows. This indicates that colour alone would not be a reliable feature for classification.
- **Lighting and Background:** Images were captured under a variety of lighting conditions and backgrounds. This background variance could lead to noise during the training if not addressed adequately. Background clutter was particularly prominent in some images.
- **Outliers:** Some images contained irrelevant or cartoon objects that did not belong to any class. Example given in the image provided which was “strawberry_0082”. These outliers could confuse the model during the training process and should be removed in preprocessing to not cause confusion for the model.



Strawberry_0082

Data Pre-processing:

To address the findings from the EDA several pre-processing steps were applied to ensure a cleaner dataset:

- **Image Resizing and Normalization:** All images were resized to a consistent dimension of 300x300 pixels to match the input requirements for CNNs and normalized to a standard range using ImageNet mean and standard deviation. This adjustment helps in stabilizing the gradients during the training and aligns with standard CNN initialization for transfer learning.
- **Data Augmentation:** To improve generalization, random horizontal flips, rotations, and slight scaled transformations were applied. By doing this, the model can learn to generalize across different orientations and scales, making it more robust to unseen data.

Methodology

Data Utilization and Splitting: The training dataset was loaded using torchvision. datstest.imageFolder, which organized them by label from the three classes. Basic data augmentation techniques were applied, such as random horizontal flipping, to enrich the dataset and reduce overfitting. Given the focus on training accuracy, I decided to use the entire dataset as a training set without splitting it into separate validation and test sets. However, while I did this the focus was on accuracy, in future iterations, cross-validation or dedicated validation set could be implemented to better estimate the model's generalization performance.

Loss Function: Cross-Entropy Loss was implemented as the objective function for training both the baseline MLP and the CNN models. Cross Entropy Loss is widely used for multi-class classification tasks as it measures the difference between the predicted and actual class probabilities, effectively penalizing incorrect predictions. Using this loss function was needed as it encourages the model to maximize the probability of the correct class for each image, directly impacting classification accuracy.

Optimization Method: The Adam Optimizer was used for training both the MLP and CNN models with a learning rate of 0.001. Adam uses adaptive learning rates and momentum, providing a balance between fast convergence and stability. Adam was used as it is computationally efficient and requires minimal tuning, which made it an attractive choice for training both models on the training dataset. Given the relatively large parameter space of CNN, Adam's adaptive nature helped achieve faster convergence compared to traditional gradient descent methods.

Regularization Strategy: For Regularization, Batch Normalization in each convolutional layer of the CNN model was used. Batch Normalization normalizes activation within each mini batch, helping to reduce internal covariate shifts allowing for more stable and faster training. Batch Normalization acts as a form of regularization by adding noise to each mini batch, which results in the reduction of overfitting.

Furthermore, transfer learning served as an implicit form of regularization in the ResNet18-based CNN model. By using pre-tuned weights, the model began with weight that were well-suited for general image feature extraction, making it less prone to overfitting on the specific fruit dataset.

Activation Function: The Rectified Linear Unit (ReLU) activation function was used in all the hidden layers of the CNN model. ReLU was chosen as it introduces non-linearity, allowing the model to learn complex patterns while avoiding the vanishing gradient problem. ReLU is computationally efficient and helps the model converge faster, making it suitable for deep networks like CNN.

Hyper-parameter Settings: For training a learning rate of 0.001 was set as a standard starting point for Adam optimization. This rate balanced training speed and stability. A Batch size of 32, which provided a good balance between computational efficiency and model performance, allowing the model to generalize better without sacrificing training speed. Lastly, 5 epochs were used for initial training assess performance.

Not explicit hyperparameter tuning was done but these settings provided a good balance for efficient training on the available dataset for the hardware capabilities available.

Data Collection and Augmentation: To improve the model's ability to generalize basic data augmentation techniques of Random Horizontal Flipping and Normalization were applied. Random Horizontal Flipping introduced variability in the dataset without altering the primary features of the images. Normalization was included to normalize the mean and standard deviation of each channel, allowing for the model to handle images with different lighting conditions more effectively. This augmentation enriched the training data implicitly, helping the model to generalize better without the need to collect additional images externally.

Transfer Learning: A pretrained model of ResNet18 was applied to the mode. The pre-trained model was initially trained on ImageNet which allowed it to learn general images features that could be fined tuned for specific tasks. Only the final fully connected layer was modified to output three classes corresponding to the dataset. To optimize the use of the pre-trained model, all layers except the final fully connected layer to be frozen, retaining learned weights for general image feature extraction and reducing the number of trainable parameters. Only the last layer was fined tuned, enabling the model to focus on specific image characteristics. This approach significantly improved classification accuracy.

Additional Techniques: The model was configured to use GPU resources if available to accelerate the training process, making it more efficient and feasible within a reasonable timeframe.

Summary and Discussion

In this project, two models were developed and evaluated. A baseline model of MLP and a best model of CNN using transfer learning with ResNet18. The MLP served as a straightforward baseline for comparison, while CNN aimed to leverage the spatial features of images to achieve higher accuracy.

The MLP model consisted of an input layer of 300 x 300 x 3 features that flattened each image, followed by one hidden layer with 512 neurons and a final output layer with 3 neurons (one per class). ReLu activation was applied in the hidden layer. The Adam optimizer was used with a learning rate of 0.001. Cross-Entropy Loss was applied to the model. It was trained for 5 epochs with a batch size of 32.

The CNN model used a pre-trained ResNet18 model as a feature extractor, which was fine-tuned for the specific classification task. Only the final fully connected layer as modified to the output classes. ReLu activation was used in convolutional layers. Adam optimizer was used with a learning rate of 0.001, fine-tuning only the final layer. Cross-Entropy Loss was applied to the model. It was trained for 5 epochs with a batch size of 32.

The MLP model, since it has fewer parameters and a simpler structure, requires little training time per epoch. In comparison, the CNN model, especially since ResNet18 was used with many layers and parameters, took significantly longer to epoch due to the complexity of the structure.

The MLP model achieves an accuracy score of approximately 44% on the training set. This highlights its limitations in handling image data. CNN on the other hand significantly outperformed the MLP with an accuracy score of approximately 72% before tuning and approximately 91% after tuning. This indicates that the CNN was able to capture the spatial relationships and patterns in the images effectively.

The CNN model demonstrates a sharp drop in loss and a steady increase in accuracy, indicating effective learning of features relevant to each class. The MLP model has a fluctuating loss with a flat accuracy trend. This further reflects its limited capacity for image classification.

The CNN outperformed the MLP by a large margin due to its convolutional layers, which are specifically designed for capturing hierarchies in image data. The MLP lacks this structure therefore struggles to learn meaningful patterns from the flattened pixel values.

The use of ResNet18 and its pre-trained data on a large diverse dataset gave CNN a strong starting point. The pre-training significantly boosted CNN's performance, making it easier to adapt to the fruit classification with fewer epochs and less fine-tuning.

The accuracy score of 90.90% from the final CNN model indicates a strong ability to generalize. CNN's architecture combined with the use of transfer learning enables it to adapt very well to the dataset, even with relatively few epochs of training.

Conclusion

The project demonstrated the effectiveness of CNNs and transfer learning for images classification tasks where visual similarity exists between classes. By leveraging spatial hierarchies, parameter efficiency, and transfer learning, CNN demonstrated a strong ability to recognize and classify patterns with a high level of accuracy (90.90%). The results show the strength of CNN architecture, especially when fine-tuned through transfer learning.

Training Baseline MLP Model:

Epoch [1/5], Loss: 0.6235, Accuracy: 39.75%
Epoch [2/5], Loss: 1.3907, Accuracy: 45.26%
Epoch [3/5], Loss: 0.9448, Accuracy: 45.64%
Epoch [4/5], Loss: 1.1300, Accuracy: 45.62%
Epoch [5/5], Loss: 0.8341, Accuracy: 44.08%
Overall Accuracy of MLP Model: 44.35%

Training tuned CNN Model:

Epoch [1/5], Loss: 1.7359, Accuracy: 88.49%
Epoch [2/5], Loss: 0.9325, Accuracy: 89.10%
Epoch [3/5], Loss: 0.9319, Accuracy: 89.92%
Epoch [4/5], Loss: 0.6388, Accuracy: 90.23%
Epoch [5/5], Loss: 0.7533, Accuracy: 90.88%
Overall Accuracy of CNN Model: 90.90%

The use of the pre-trained ResNet18 model for transfer learning proved to be an efficient and effective approach, allowing the network to converge quickly and achieve high performance with limited training. This approach minimized the need for extensive computational resources and large datasets. Additionally, regularization techniques such as batch normalization contributed to the model's robustness and ability to generalize, addressing potential overfitting concerns.

Future work: Currently, only the final layer of ResNet18 was fine-tuned. Future work could involve unfreezing and fine-tuning additional layers to allow the model to learn task specific features more effectively. However, this would require very careful monitoring to make sure there is no overfitting.

Furthermore, to improve generalization, data augmentation techniques like random rotations, scaling, colour jittering, and translations could be applied. These augmentations would increase the effective size and diversity of the dataset, helping the model become more robust.

Although ResNet18 performed well, a custom-built CNN could be designed specifically to capture unique features of fruits. Custom architecture could include tailored layers or specialized pooling strategies that emphasize shape and texture distinctions in the fruit images.